

More on Deep Learning

# Networks Using Blocks

---

- While AlexNet offered empirical evidence that deep CNNs can achieve good results, it did not provide a general template to guide subsequent researchers in designing new networks.
- Progress in this field mirrors that in chip design where engineers went from placing transistors to logical elements to logic blocks.
- Similarly, the design of neural network architectures had grown progressively more abstract, with researchers moving from thinking in terms of individual neurons to whole layers, and to blocks, repeating patterns of layers.

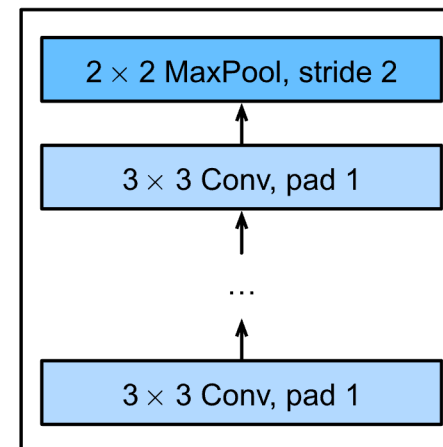
# VGG Net

---

- The idea of using blocks first emerged from VGG network.
- The basic building block is a sequence of the following:
  - Convolutional layer with padding;
  - Nonlinearity such as ReLU;
  - Pooling layer such as max pooling.

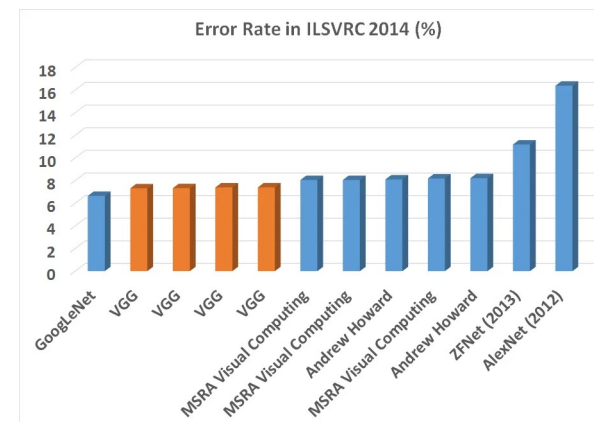
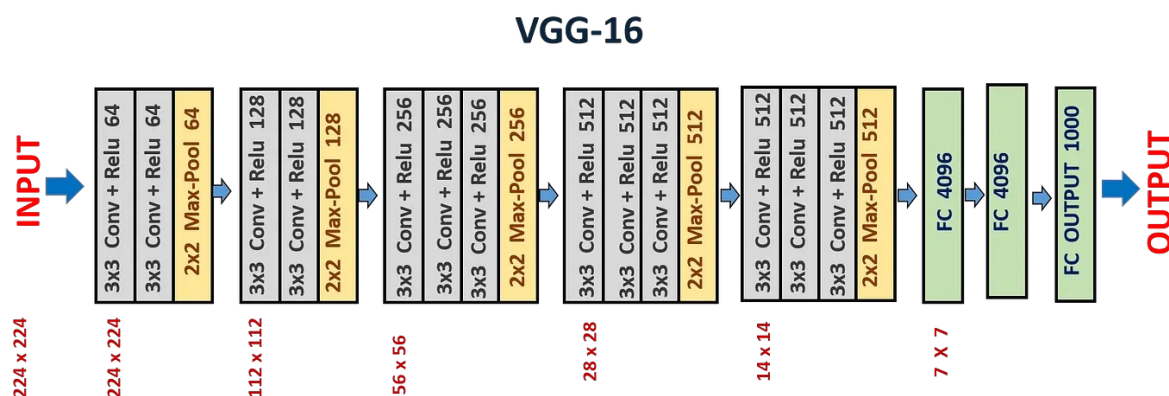
```
def vgg_block(num_convs, in_channels, out_channels):  
    layers=[]  
    for _ in range(num_convs):  
        layers.append(nn.Conv2d(in_channels, out_channels,  
                                kernel_size=3, padding=1))  
        layers.append(nn.ReLU())  
        in_channels = out_channels  
    layers.append(nn.MaxPool2d(kernel_size=2, stride=2))  
    return nn.Sequential(*layers)
```

Example code for VGG block



# VGG Net

- The VGG Network can be partitioned into two parts:
  - The first part consists mostly of convolutional and pooling layers;
    - Connecting several VGG blocks in succession.
  - The second part consists of fully-connected layers.



# Network in Network (NiN)

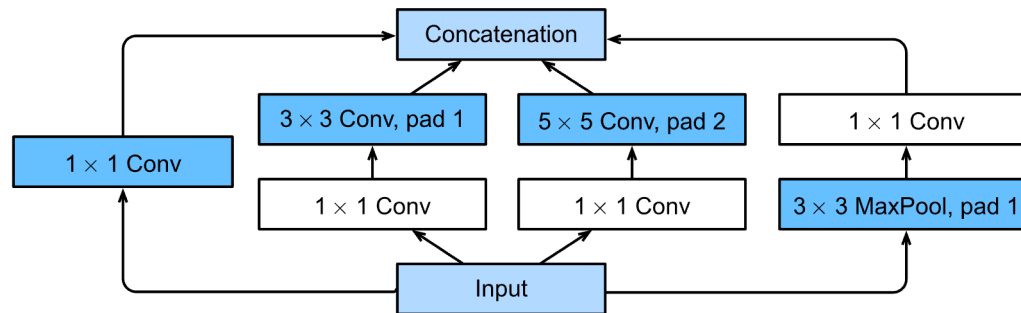
---

- NiN uses blocks consisting of a convolutional layer and multiple  $1 \times 1$  convolutional layers.
  - This can be used within the convolutional stack to allow for more per-pixel nonlinearity.
- NiN removes the fully-connected layers and replaces them with global average pooling (i.e., summing over all locations) after reducing the number of channels to the desired number of outputs.
  - Removing the fully-connected layers reduces number of parameters and risk of overfitting.

# GoogLeNet

---

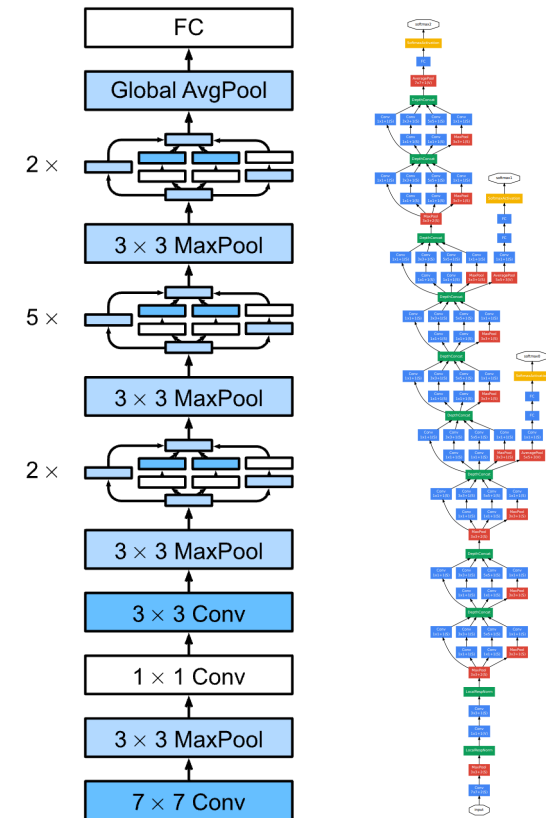
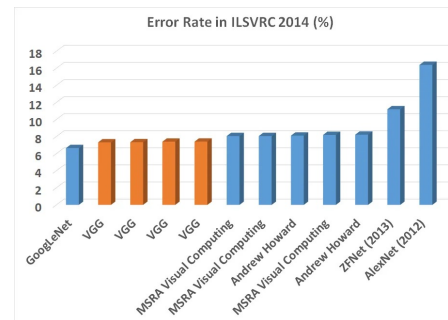
- In 2014, GoogLeNet won the ImageNet Challenge.
  - Use the paradigm of repeated NiN blocks;
  - Introduce the inception block that combines variously sized kernels in parallel.



Inception Block

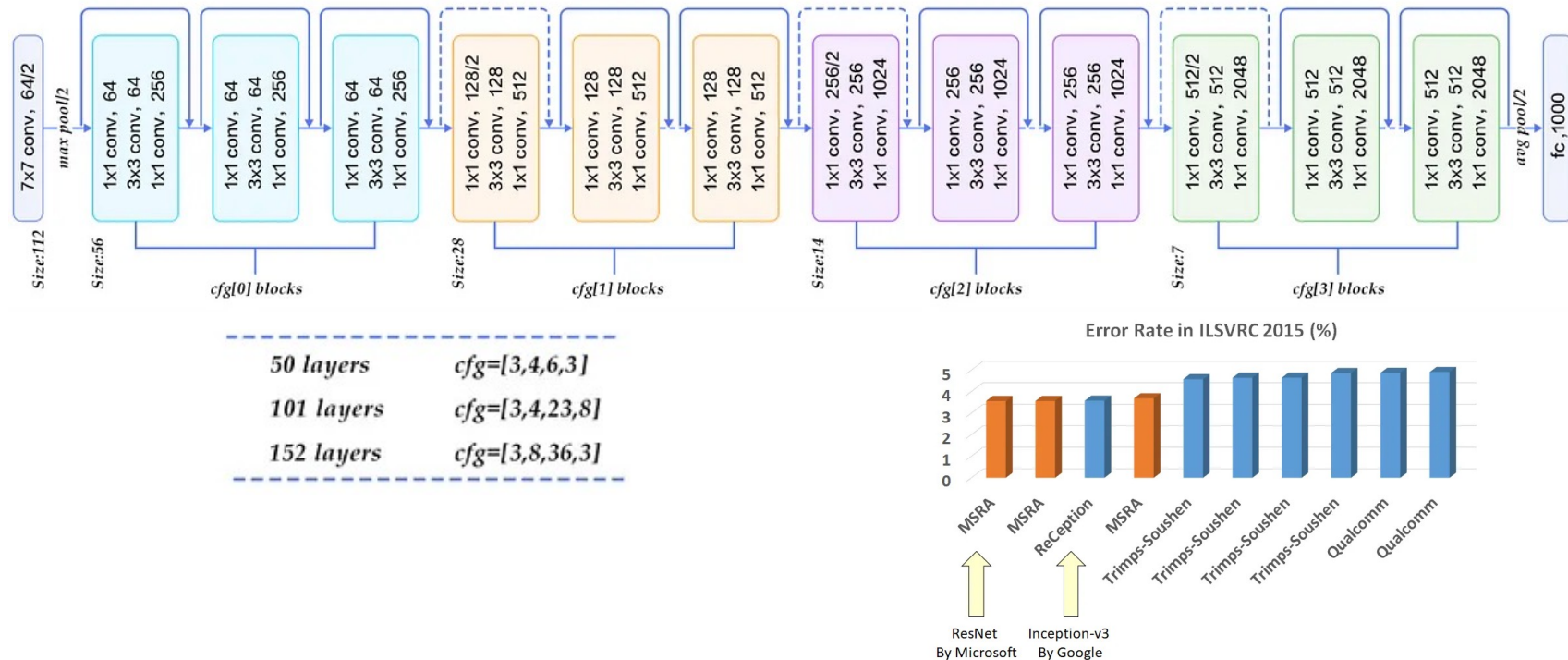
# GoogLeNet

- In 2014, GoogLeNet won the ImageNet Challenge.
  - It uses a stack of a total of 9 inception blocks and global average pooling to generate its estimates.
  - The Inception block is equivalent to a subnetwork with four paths.
  - It is one of the most efficient models on ImageNet, providing similar test accuracy with lower computational complexity.



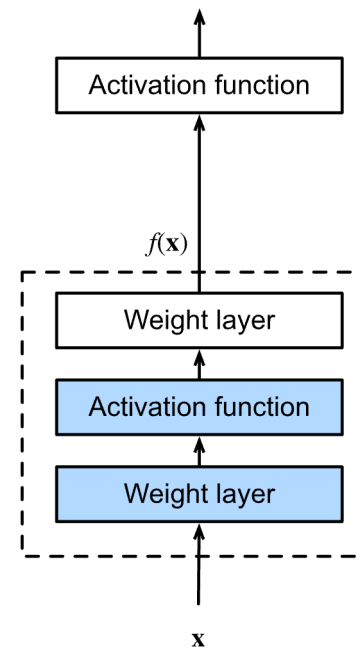
# Residual Networks (ResNet)

- In 2015, ResNet (152 layers) won the ImageNet Challenge.

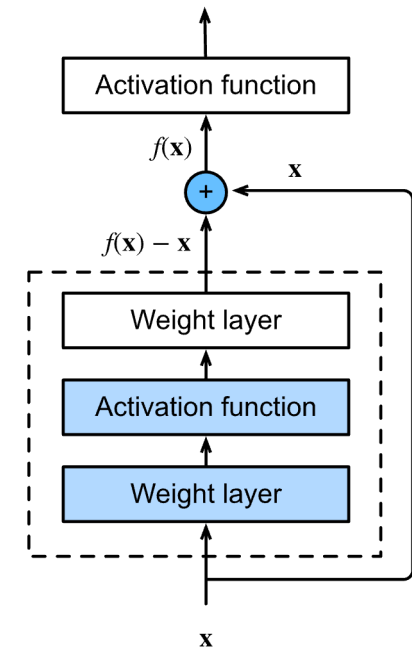


# ResNet

- **Residual learning**: allows the network to train very deep architecture effectively, addressing the problem of vanishing and exploding gradients that can occur in traditional deep neural networks.
  - On the left, the portion within the dotted-line box must directly learn the **direct mapping**  $f(x)$ .
  - On the right, the portion within the dotted-line box needs to learn the **residual mapping**  $f(x) - x$ .



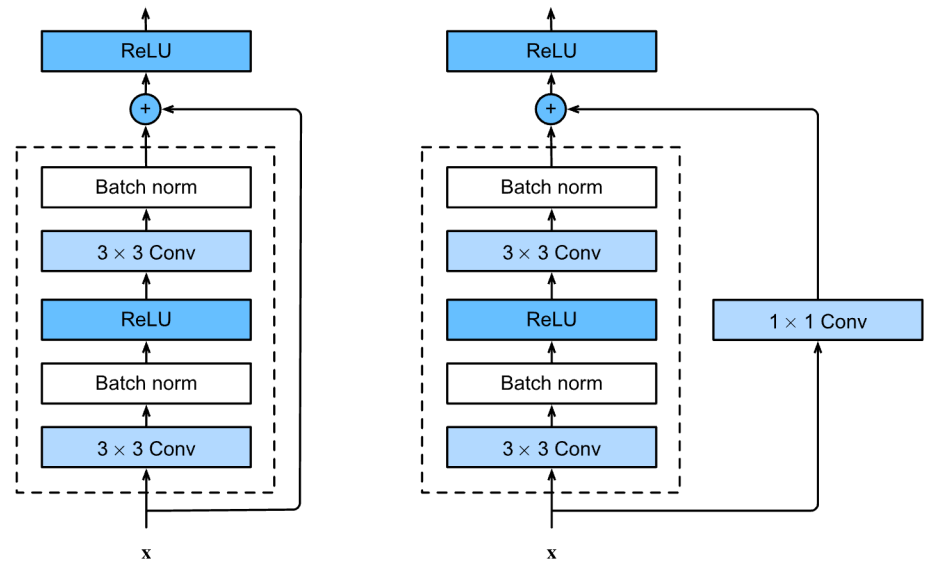
Traditional block



Residual block

# Residual Blocks

- Shortcut Connections:
  - Identity mappings are added to the outputs of stacked layers.
  - By directly allowing gradients to flow through these shortcut paths, deeper networks can effectively and efficiently propagate gradients back through earlier layers during training.



# Batch Normalization

---

- Batch normalization is a powerful technique to accelerate convergence of deep networks in training.
  - In each training iteration, we first normalize the inputs (of batch normalization) by subtracting their mean and dividing by their standard deviation, where both are estimated based on the statistics of the current minibatch.
  - Next, we apply a scale coefficient and a scale offset, which are learnable.

$$\text{BN}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \hat{\boldsymbol{\mu}}_B}{\hat{\boldsymbol{\sigma}}_B} + \beta.$$
$$\hat{\boldsymbol{\mu}}_B = \frac{1}{|B|} \sum_{\mathbf{x} \in B} \mathbf{x},$$
$$\hat{\boldsymbol{\sigma}}_B^2 = \frac{1}{|B|} \sum_{\mathbf{x} \in B} (\mathbf{x} - \hat{\boldsymbol{\mu}}_B)^2 + \epsilon.$$

Sample mean of the minibatch

Standard deviation of the minibatch

# Example: ResNet-18

```
b1 = nn.Sequential(nn.Conv2d(1, 64, kernel_size=7, stride=2, padding=3),
                  nn.BatchNorm2d(64), nn.ReLU(),
                  nn.MaxPool2d(kernel_size=3, stride=2, padding=1))
```

```
b2 = nn.Sequential(*resnet_block(64, 64, 2, first_block=True))
```

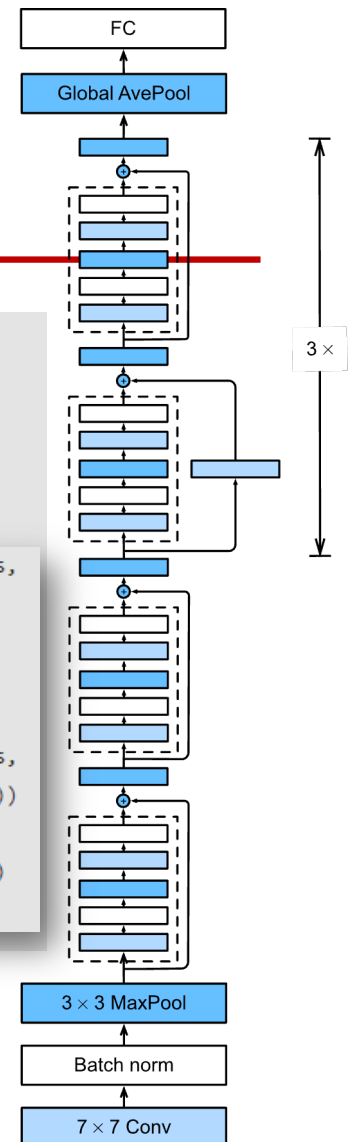
```
b3 = nn.Sequential(*resnet_block(64, 128, 2))
```

```
b4 = nn.Sequential(*resnet_block(128, 256, 2))
```

```
b5 = nn.Sequential(*resnet_block(256, 512, 2))
```

```
net = nn.Sequential(b1, b2, b3, b4, b5,
                  nn.AdaptiveAvgPool2d((1,1)),
                  nn.Flatten(), nn.Linear(512, 10))
```

```
def resnet_block(input_channels, num_channels, num_residuals,
                 first_block=False):
    blk = []
    for i in range(num_residuals):
        if i == 0 and not first_block:
            blk.append(Residual(input_channels, num_channels,
                                use_1x1conv=True, strides=2))
        else:
            blk.append(Residual(num_channels, num_channels))
    return blk
```



## Cont. on CNN

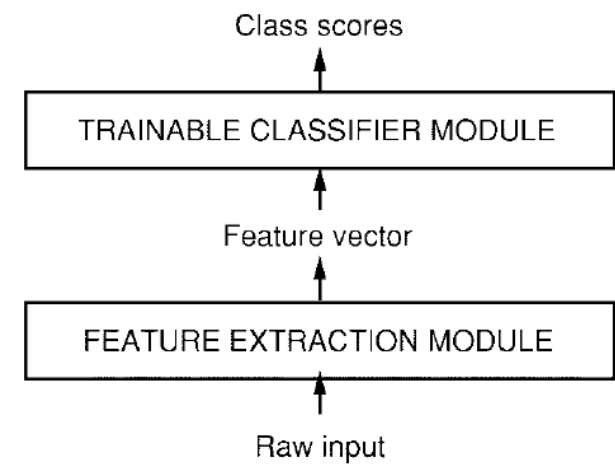
---

- We have seen typical CNN architectures.
  - VGGNet: Known for its simplicity and use of very small (3x3) convolution filters with a deep architecture.
  - Inception (GoogLeNet): Utilizes a module that applies multiple convolution filters of different sizes in parallel, capturing a variety of spatial features.
  - ResNet: Incorporates skip connections, allowing training of very deep networks by mitigating vanishing gradient problems.
  - DenseNet: Every layer receives inputs from all previous layers, promoting feature reuse and reducing the number of parameters.
  - MobileNet: Designed for efficiency, using depthwise separable convolutions suitable for mobile and embedded devices.

## Cont. on CNN

---

- **Backbone**: Refers to the core part of a neural network architecture.
  - Mainly responsible for **feature extraction** from input images;
  - Composed of multiple layers that progressively transform the input image into high-level feature maps;
  - Including convolutional layers, activation functions, pooling layers, and sometimes normalization layers.



From LeNet's paper in 1998

# More than Classification

- But there are many types of applications other than classification only.
  - Such as face detection (e.g., by the Viola-Jones method).

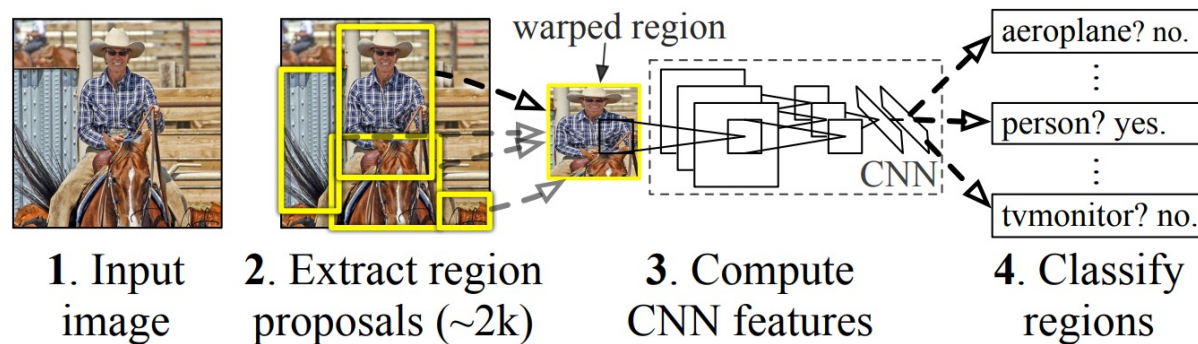


Viola-Jones method: Haar-like features + Boosting for classification

# RCNN for Detection

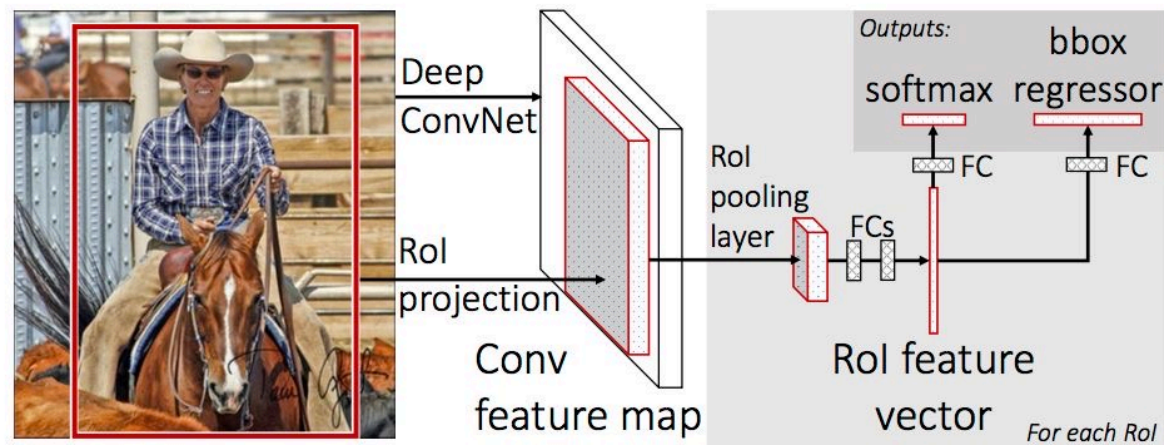
---

- **RCNN**, which stands for Region-based Convolutional Neural Network, is a first try for localizing objects in images via deep learning.
  - **Region proposal**: Generate a large number of region proposals (e.g., ~2000), where each of these region proposals might potentially contain an object.
  - **Feature extraction**: Each region is then reshaped to a fixed size and passed through a CNN (e.g., AlexNet) to extract features.
  - **Classification**: The features extracted from each region are fed into a set of classifiers (e.g., linear SVMs) for classification.



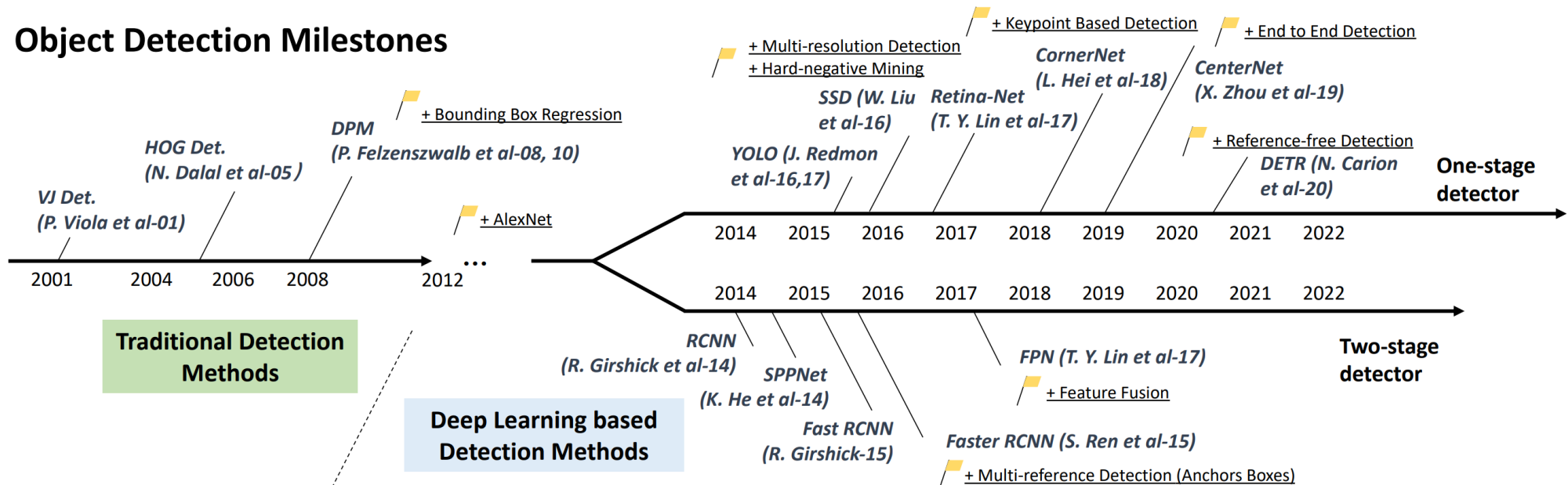
# Fast-RCNN

- RCNN is slow, so improvement on speed is expected, which leads to Fast-RCNN.
  - The entire image is passed through a CNN once to produce a **single feature map**.
  - **Region of Interest (RoI) pooling** is applied to extract fixed-size feature vectors from the feature map for each region proposal.
  - A **softmax layer** is used for classification, predicting scores for each class. Simultaneously, bounding box regression is performed for refining the coordinates of the proposals.

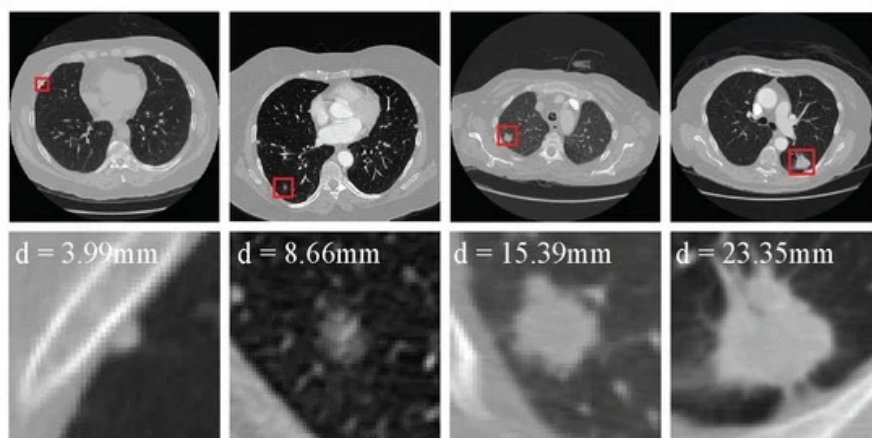


# Roadmap of Object Detection

## Object Detection Milestones



# Example: Lung Nodule Detection



本指导原则是对肺结节CT图像辅助检测软件的一般要求，申请人应依据产品的具体特性确定其中的内容是否适用。若不适用，需具体阐述其理由及相应的科学依据，并依据具体的产品特性对注册申报资料的内容进行充实和细化。

本指导原则是供注册申请人和技术审评人员使用的指导性文件，但不包括注册审批所涉及的行政事项，亦不作为法规强制执行，应在遵循相关法规的前提下使用本指导原则。如果有能够满足相关法规要求的其他方法，也可以采用，但是需要提供详细的研究资料和验证资料。

本指导原则是在现行法规和标准体系以及当前认知水平下制定的，随着法规和标准的不断完善，以及科学技术的不断发展，相关内容也将适时进行调整。

本指导原则是人工智能医疗器械指导原则体系的重要组成部分，基于人工智能医疗器械审评指导原则的通用要求，明确了肺结节CT图像辅助检测软件的具体要求。

## 一、适用范围

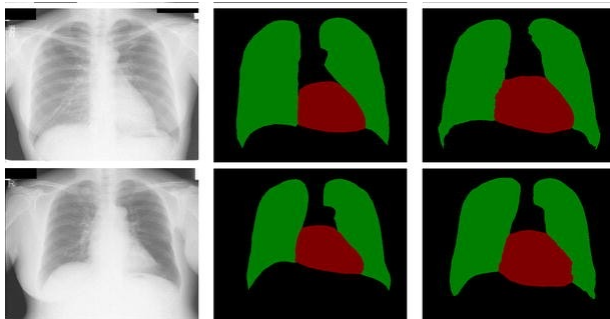
本指导原则适用于肺结节CT图像辅助检测软件的产品注册。按现行《医疗器械分类目录》，该类产品分类编码为21-04-02，管理类别为三类。设备中所含肺结节CT图像辅助检测功能也适用于本指导原则。采用非人工智能算法的肺结节CT图像辅助检测软件可参照本指导原则执行。

《肺结节CT图像辅助检测软件注册审查指导原则》

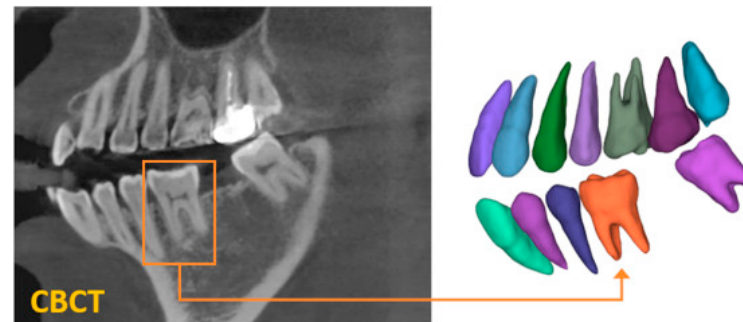
# Fully Convolutional Network (FCN)

---

- Fully Convolutional Networks (FCNs) are designed specifically for dense prediction tasks, such as image segmentation.
  - Unlike traditional CNNs used for image classification, FCNs don't have any fully connected layers; instead, they consist entirely of convolutional and pooling layers.
  - This architecture enables them to output a spatial map rather than a single label per image, making them ideal for tasks where the output needs to correspond to the input dimensionally, such as predicting the category of each pixel in an image.



Semantic segmentation



Instance segmentation

# U-Net

- U-Net is a type of FCN that was specifically designed for medical image segmentation.
  - It's known for its **U-shaped architecture**, featuring a strong ability to localize and delineate structures of interest with a small number of training images.
  - With effectiveness in capturing fine-grained details and large context simultaneously, U-Net and its variants are successfully applied in various domains and tasks.

## SPRINGER NATURE Link

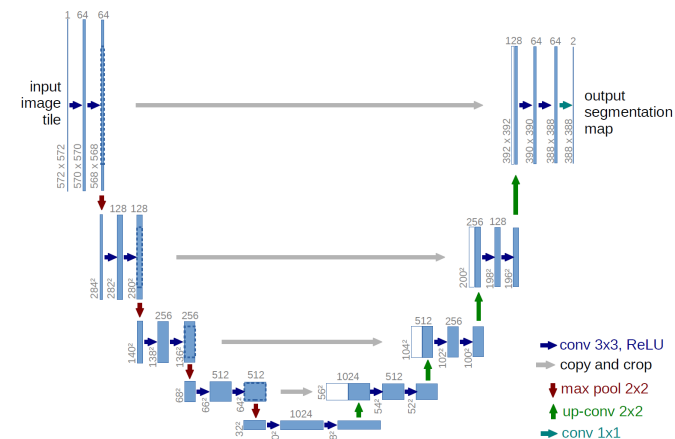
Find a journal Publish with us Track your research Search

Home > Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015 > Conference paper

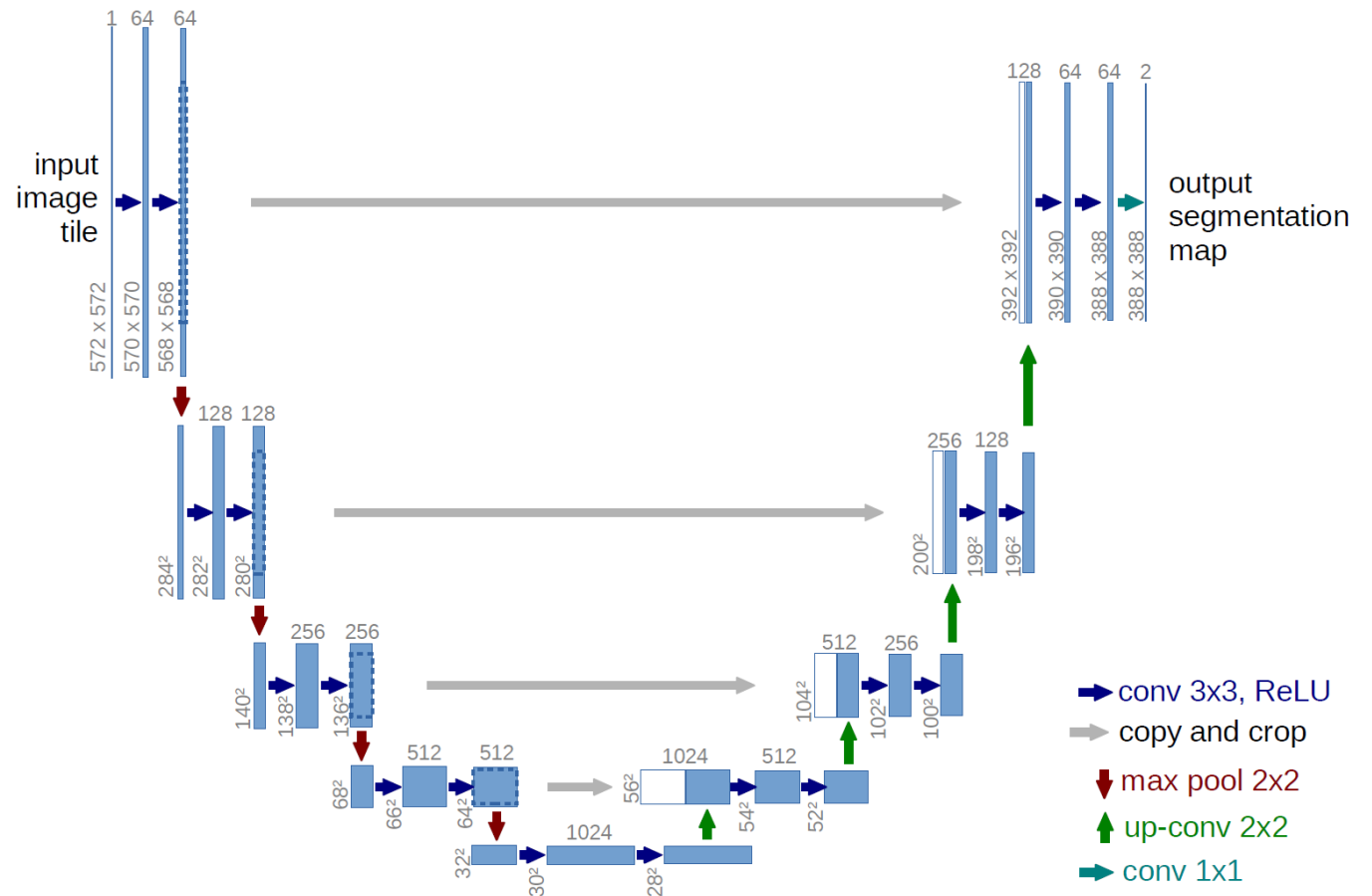
### U-Net: Convolutional Networks for Biomedical Image Segmentation

Conference paper | First Online: 18 November 2015

pp 234–241 | [Cite this conference paper](#)

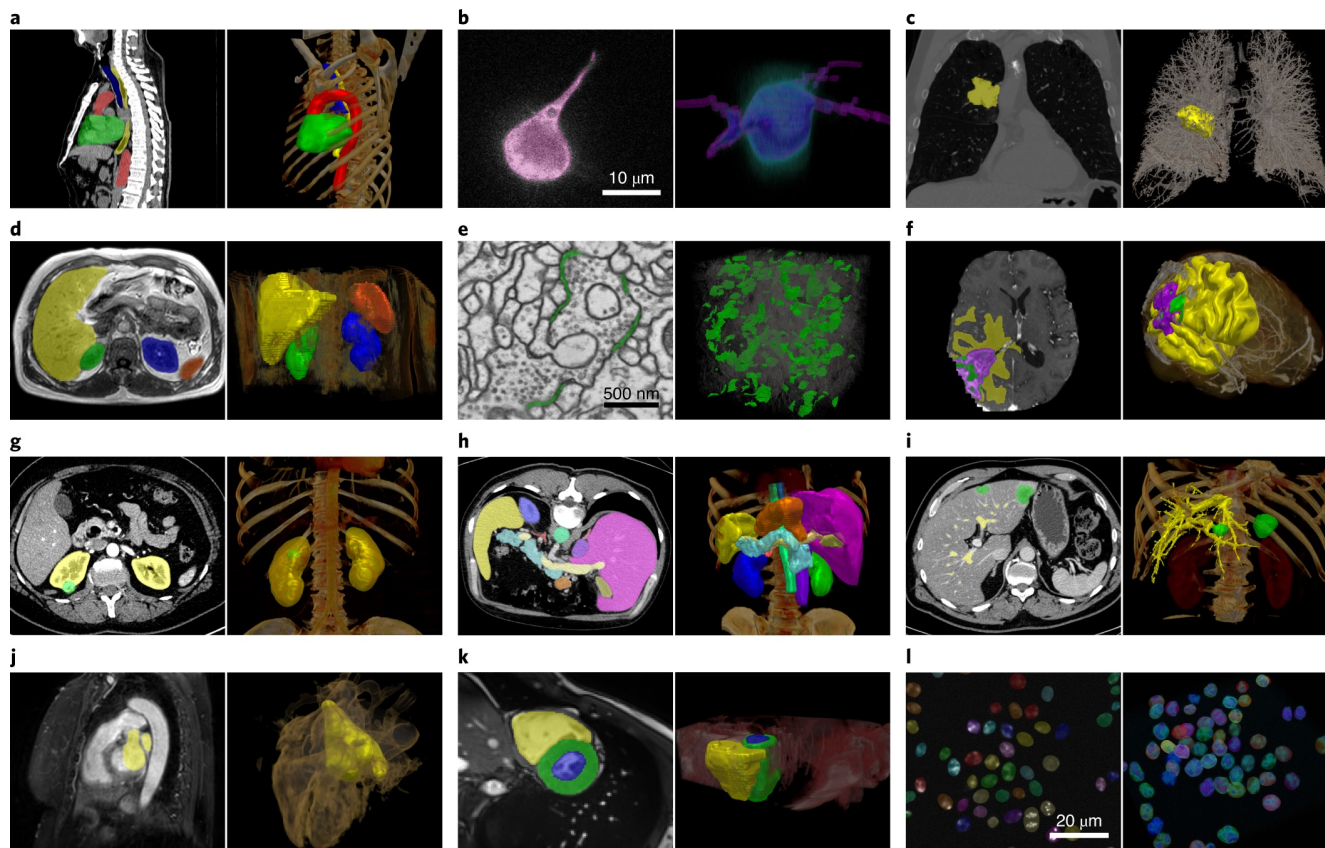


- U-shaped architecture;
- Convolutional layers only;
- Encoder path;
- Decoder path;
- Skip connections.



# nnU-net

---



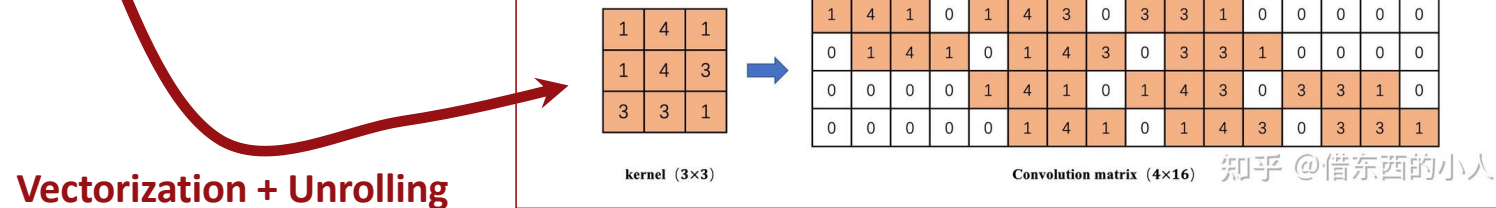
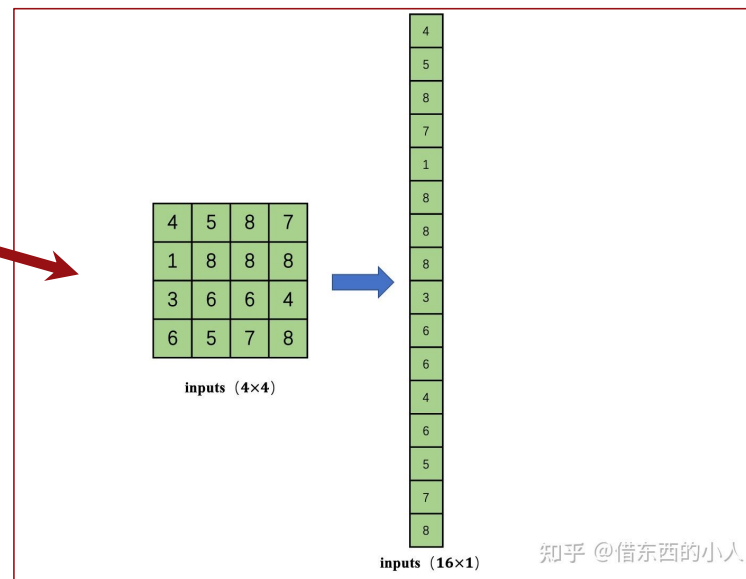
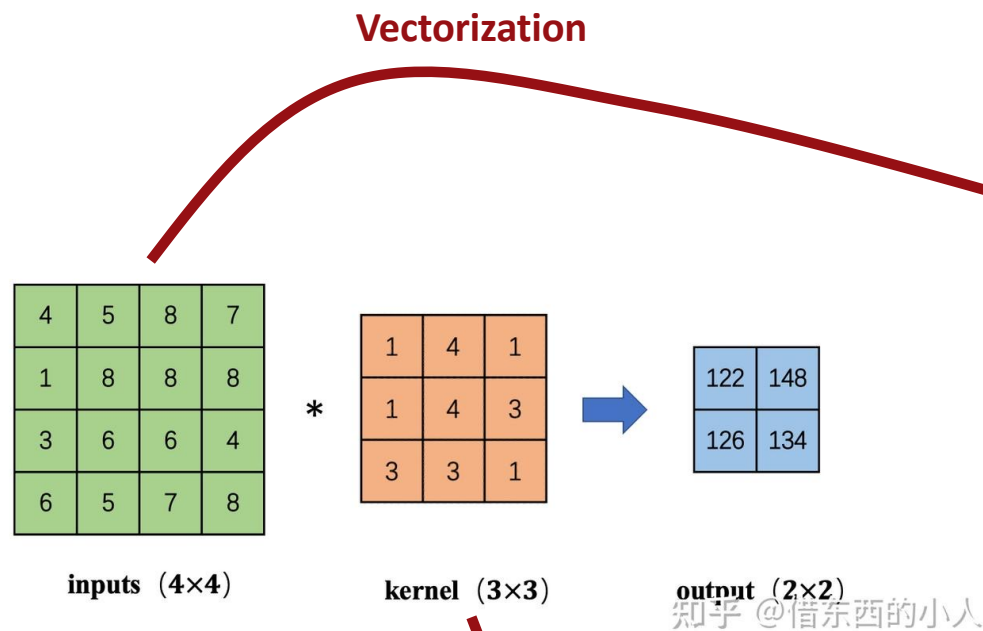
# Rethink on Convolution

---

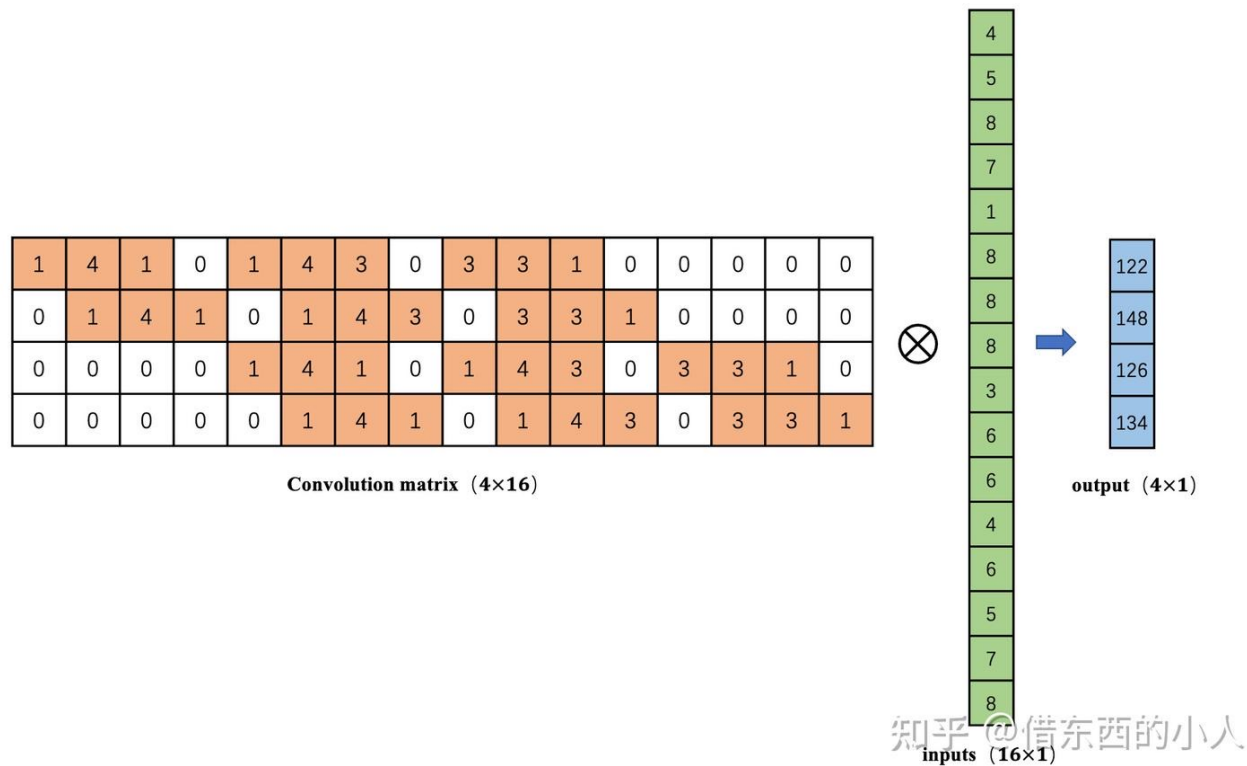
- All convolutional operations (and many others) we have seen tend to reduce feature map size.
  - That's why we need padding to keep feature size unchanged.

| Input                                                                                                                                           |    | Kernel |   | Output |   |   |   |   |   |   |                                                                                      |   |   |   |   |   |                                                                                          |    |    |    |    |
|-------------------------------------------------------------------------------------------------------------------------------------------------|----|--------|---|--------|---|---|---|---|---|---|--------------------------------------------------------------------------------------|---|---|---|---|---|------------------------------------------------------------------------------------------|----|----|----|----|
| <table border="1"><tr><td>0</td><td>1</td><td>2</td></tr><tr><td>3</td><td>4</td><td>5</td></tr><tr><td>6</td><td>7</td><td>8</td></tr></table> | 0  | 1      | 2 | 3      | 4 | 5 | 6 | 7 | 8 | * | <table border="1"><tr><td>0</td><td>1</td></tr><tr><td>2</td><td>3</td></tr></table> | 0 | 1 | 2 | 3 | = | <table border="1"><tr><td>19</td><td>25</td></tr><tr><td>37</td><td>43</td></tr></table> | 19 | 25 | 37 | 43 |
| 0                                                                                                                                               | 1  | 2      |   |        |   |   |   |   |   |   |                                                                                      |   |   |   |   |   |                                                                                          |    |    |    |    |
| 3                                                                                                                                               | 4  | 5      |   |        |   |   |   |   |   |   |                                                                                      |   |   |   |   |   |                                                                                          |    |    |    |    |
| 6                                                                                                                                               | 7  | 8      |   |        |   |   |   |   |   |   |                                                                                      |   |   |   |   |   |                                                                                          |    |    |    |    |
| 0                                                                                                                                               | 1  |        |   |        |   |   |   |   |   |   |                                                                                      |   |   |   |   |   |                                                                                          |    |    |    |    |
| 2                                                                                                                                               | 3  |        |   |        |   |   |   |   |   |   |                                                                                      |   |   |   |   |   |                                                                                          |    |    |    |    |
| 19                                                                                                                                              | 25 |        |   |        |   |   |   |   |   |   |                                                                                      |   |   |   |   |   |                                                                                          |    |    |    |    |
| 37                                                                                                                                              | 43 |        |   |        |   |   |   |   |   |   |                                                                                      |   |   |   |   |   |                                                                                          |    |    |    |    |

- U-net is a fully convolutional network, yet we need upsample feature size.
- We know MLP layer can be differentiated, but how is convolutional layer?

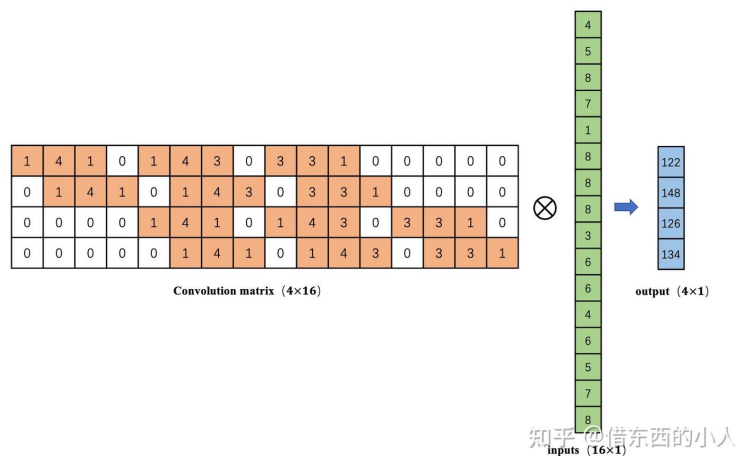


# Convolution in Matrix Form

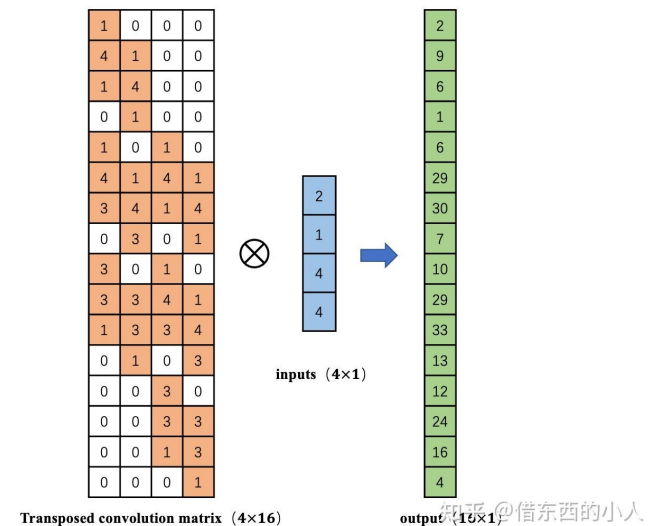


# Transposed Convolution

- Transposed convolution: Upsamples lower-resolution feature maps to higher resolutions, essentially performing the opposite of a standard convolution.



Standard convolution



Transposed convolution

|   |   |   |   |
|---|---|---|---|
| 1 | 0 | 0 | 0 |
| 4 | 1 | 0 | 0 |
| 1 | 4 | 0 | 0 |
| 0 | 1 | 0 | 0 |
| 1 | 0 | 1 | 0 |
| 4 | 1 | 4 | 1 |
| 3 | 4 | 1 | 4 |
| 0 | 3 | 0 | 1 |
| 3 | 0 | 1 | 0 |
| 3 | 3 | 4 | 1 |
| 1 | 3 | 3 | 4 |
| 0 | 1 | 0 | 3 |
| 0 | 0 | 3 | 0 |
| 0 | 0 | 3 | 3 |
| 0 | 0 | 1 | 3 |
| 0 | 0 | 0 | 1 |

Transposed convolution matrix (4×16)



|   |
|---|
| 2 |
| 1 |
| 4 |
| 4 |

inputs (4×1)



|    |
|----|
| 2  |
| 9  |
| 6  |
| 1  |
| 6  |
| 29 |
| 30 |
| 7  |
| 10 |
| 29 |
| 33 |
| 13 |
| 12 |
| 24 |
| 16 |
| 4  |

output (16×1)

|   |   |   |
|---|---|---|
| 1 | 4 | 1 |
| 1 | 4 | 3 |
| 3 | 3 | 1 |

旋转180°

|   |   |   |
|---|---|---|
| 1 | 3 | 3 |
| 3 | 4 | 1 |
| 1 | 4 | 2 |
|   |   | 1 |
|   |   | 4 |
|   |   | 4 |

|   |   |   |
|---|---|---|
| 1 | 4 | 1 |
| 1 | 4 | 3 |
| 3 | 3 | 1 |

旋转180°

|   |   |   |
|---|---|---|
| 1 | 3 | 3 |
| 3 | 4 | 1 |
| 1 | 4 | 2 |
|   |   | 1 |
|   |   | 4 |
|   |   | 4 |

|   |   |   |
|---|---|---|
| 1 | 4 | 1 |
| 1 | 4 | 3 |
| 3 | 3 | 1 |

旋转180°

|   |   |   |
|---|---|---|
| 1 | 3 | 3 |
| 3 | 4 | 1 |
|   |   | 1 |
|   |   | 4 |
|   |   | 1 |
|   |   | 4 |

输入

|   |   |
|---|---|
| 2 | 1 |
| 4 | 4 |

卷积核

|   |   |   |
|---|---|---|
| 1 | 4 | 1 |
| 1 | 4 | 3 |
| 3 | 3 | 1 |

知乎 @借东西的小人

# Summary: CNN

---

- CNN is a network that employs convolutional layers.
  - Many network architectures have emerged after AlexNet.
  - In a CNN, we interleave convolutions, nonlinearities, and (often) pooling operations.
  - Convolutional layers are typically arranged so that they gradually decrease the spatial resolution of the representations, while increasing the number of channels.
    - The output of activations from convolutional operations is called feature map.
- The power of CNNs lies in their ability to automatically learn multiple layers of feature hierarchies, making them highly effective for visual processing tasks.
- Usually a network's success relies not only on its architecture, but many implementation issues in detail as well.

# Discriminative vs. Generative Models

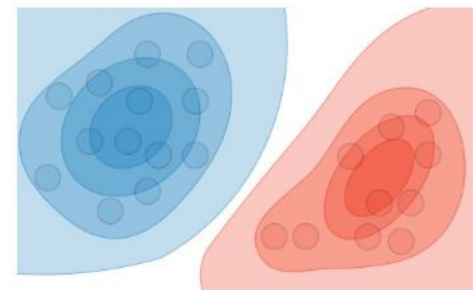
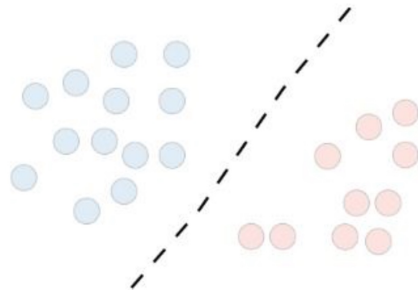
---

- Discriminative models:
  - Focus on modeling the **decision boundary** between classes rather than the distribution of individual classes.
  - Early CNNs are a prime example of discriminative models; they excel at tasks like image classification by learning hierarchical feature representations that effectively delineate between different classes.
- Generative Models:
  - Aim to model the **joint probability distribution  $P(x|y)$**  capturing how data points are distributed across classes.
  - These models can **generate new data samples** given some conditions.
  - Bayesian Networks, in particular, provide a probabilistic graphical model that represents a set of variables and their conditional dependencies via a directed acyclic graph.

# Discriminative vs. Generative Models

|                 | Discriminative learning    | Generative learning                       |
|-----------------|----------------------------|-------------------------------------------|
| Goal            | Directly estimate $p(y x)$ | Estimate $p(x y)$ to then reduce $p(y x)$ |
| What's learned? | Decision boundary          | Probability distributions of the data     |

Illustration



$$p(y|x) = \frac{p(y) \cdot p(x|y)}{p(x)}$$

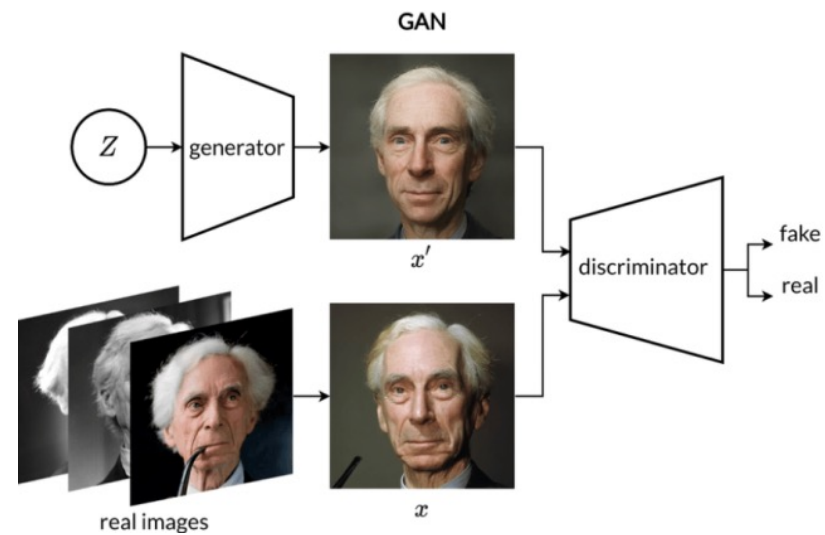
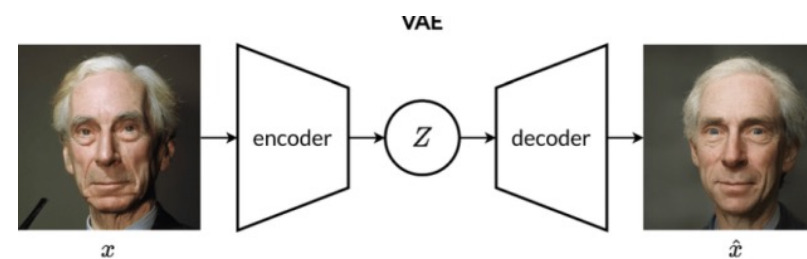
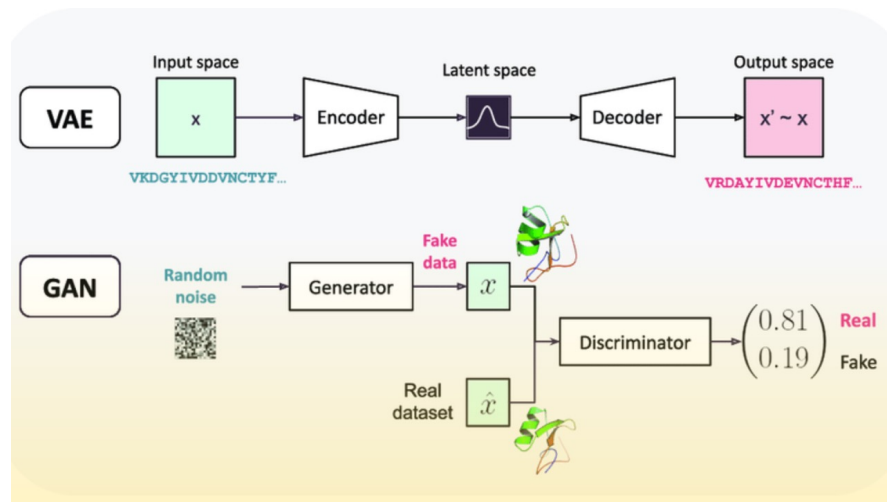
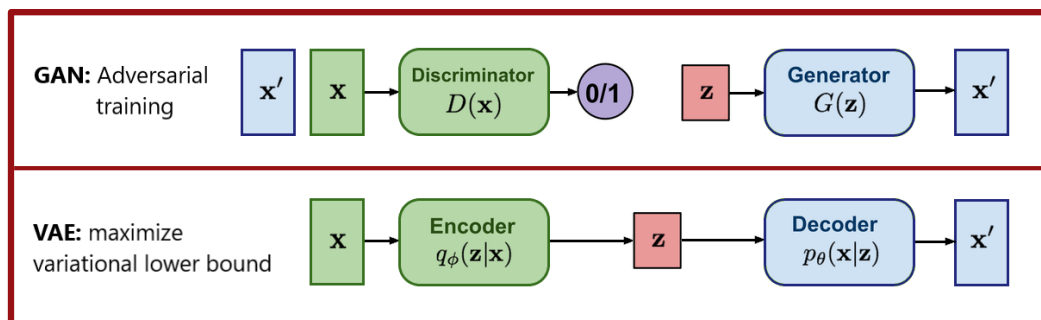
$$\textit{Posterior} = \frac{\textit{Prior} \cdot \textit{Likelihood}}{\textit{Evidence}}$$

# Deep Generative Model

---

- With the advent of deep learning, new types of generative models have been developed.
  - GAN (Generative Adversarial Network):
    - Comprising a generator and discriminator, GANs have revolutionized generative tasks by providing a framework for generating highly realistic data samples, particularly images.
  - VAE (Variational Autoencoders):
    - Combine concepts from both discriminative (autoencoders) and generative modeling, allowing for efficient latent space learning and generation of new samples.
  - Normalizing Flow, Autoregressive Model, Transformer-Based Model, Diffusion Model, Neural Radiance Fields, etc.

# Deep Generative Model

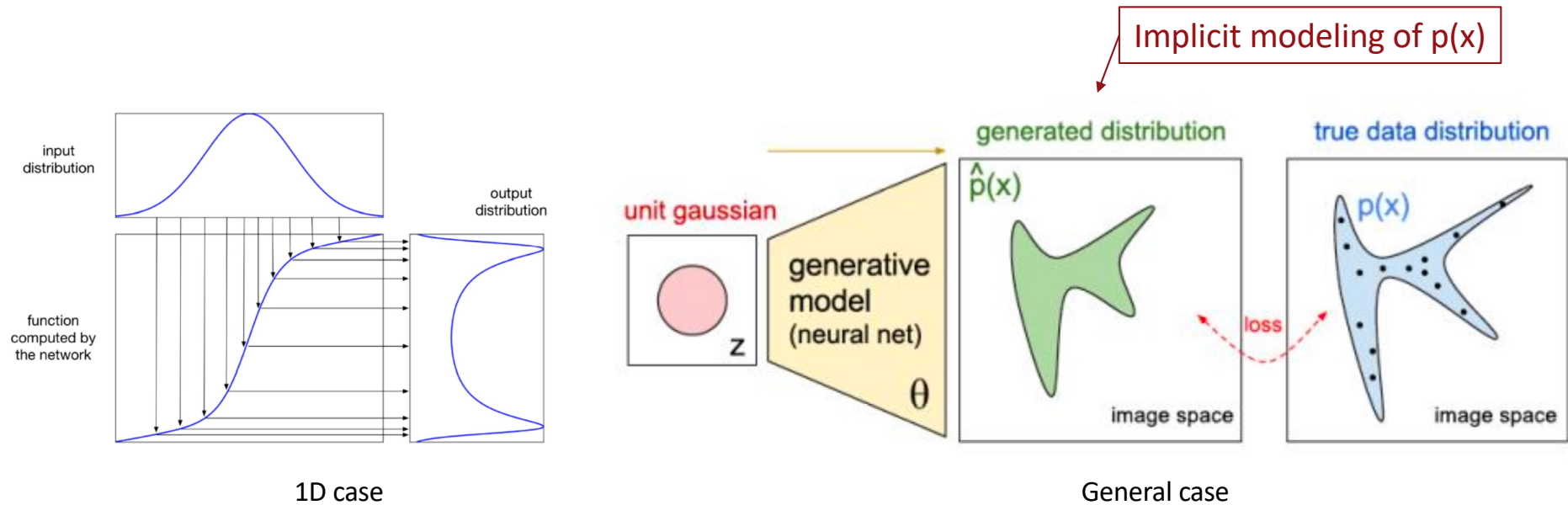


# GAN

---

- GANs consist of two neural networks that are trained simultaneously with opposing goals: the generator and the discriminator.
  - Generator:
    - The generator's objective is to create data that is indistinguishable from real data.
    - It takes random noise as input and transforms it into a data sample (e.g., an image).
    - Over the course of its training, the generator learns the distribution of the training data so it can more convincingly produce samples that resemble the true data.
  - Discriminator:
    - The discriminator's role is to distinguish between real data and the data produced by the generator.
    - It outputs a probability indicating whether a given input is real or fake.

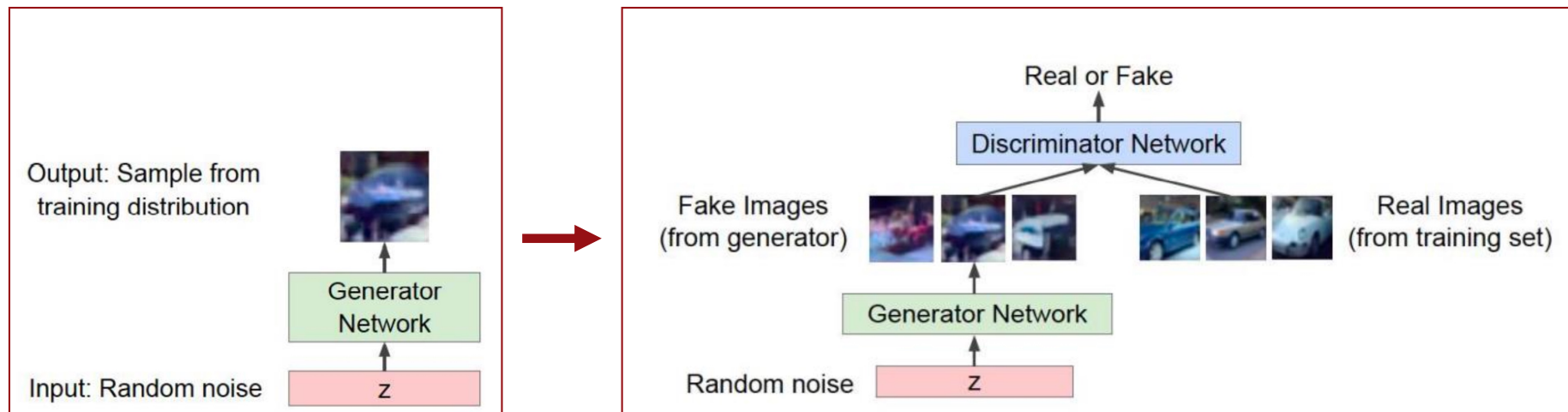
# GAN: Generator



Generator: mapping from noise to data

# GAN: Generator-Discriminator

---



# GAN: Adversarial Training

---

- The generator and discriminator engage in a **two-player minimax game**, where the generator tries to fool the discriminator, and the discriminator tries to accurately classify inputs as real or fake.
  - The **generator** aims to **minimize** the log-likelihood of the discriminator's assessment that a generated instance is fake.
  - Conversely, the **discriminator** aims to **maximize** its ability to correctly classify both real data as real and generated data as fake.

# GAN: Adversarial Training

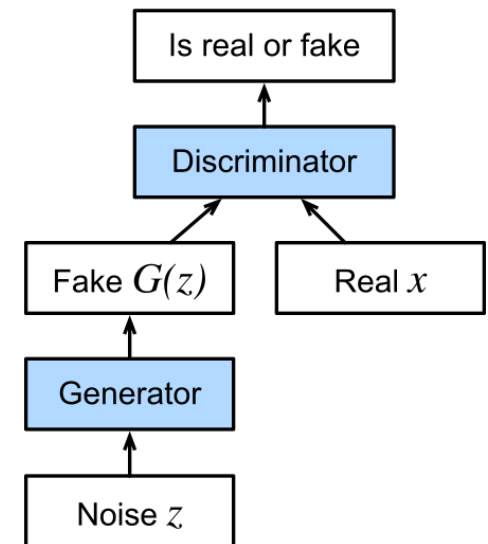
---

1. Sample random noise.
2. Produce generator output from sampled random noise.
3. Get discriminator "Real" or "Fake" classification for generator output.
4. Calculate loss from discriminator classification.

The generator tries to minimize while the discriminator tries to maximize:

$$E_x[\log(D(x))] + E_z[\log(1 - D(G(z)))]$$

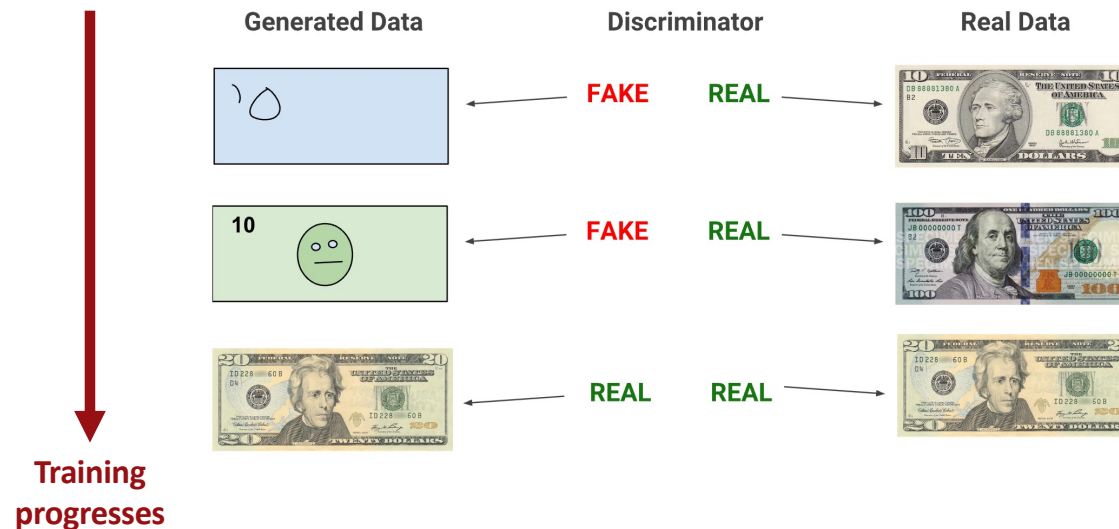
5. Backpropagate through both the discriminator and generator to obtain gradients.
6. Use gradients to change only the generator weights.



We assume "real" is indicated by "1" and "fake" is for "0".

# GAN: Adversarial Training

- The training process **alternates** between updating the **discriminator** and the **generator**.
- Still, training a GAN is often very difficult.
  - If your discriminator is too good, then generator training can fail due to vanishing gradients.
  - Mode collapse: generator produces a limited variety of outputs to fool discriminator.



# Wasserstein GAN (WGAN)

---

- Wasserstein Distance is a measure of the distance between two probability distributions.
  - It conceptually represents the minimum cost of transforming one distribution into the other.
- Use “Critic” network ( $f$ ) to replace Discriminator ( $D$ ).
  - The output of critic is a real number not limited to  $[0, 1]$ , approximating the “realness” score (which reflects the Wasserstein distance to certain extent).

The “critic” learns to maximize the loss function configured as measuring the Wasserstein distance between “real” and “generated”

$$L(p_r, p_g) = W(p_r, p_g) = \max_{w \in W} \mathbb{E}_{x \sim p_r} [f_w(x)] - \mathbb{E}_{z \sim p_r(z)} [f_w(g_\theta(z))]$$

Alternatively, the “generator” wants to maximize this term.

# WGAN

---

---

**Algorithm 1** WGAN, our proposed algorithm. All experiments in the paper used the default values  $\alpha = 0.00005$ ,  $c = 0.01$ ,  $m = 64$ ,  $n_{\text{critic}} = 5$ .

---

**Require:** :  $\alpha$ , the learning rate.  $c$ , the clipping parameter.  $m$ , the batch size.  
 $n_{\text{critic}}$ , the number of iterations of the critic per generator iteration.

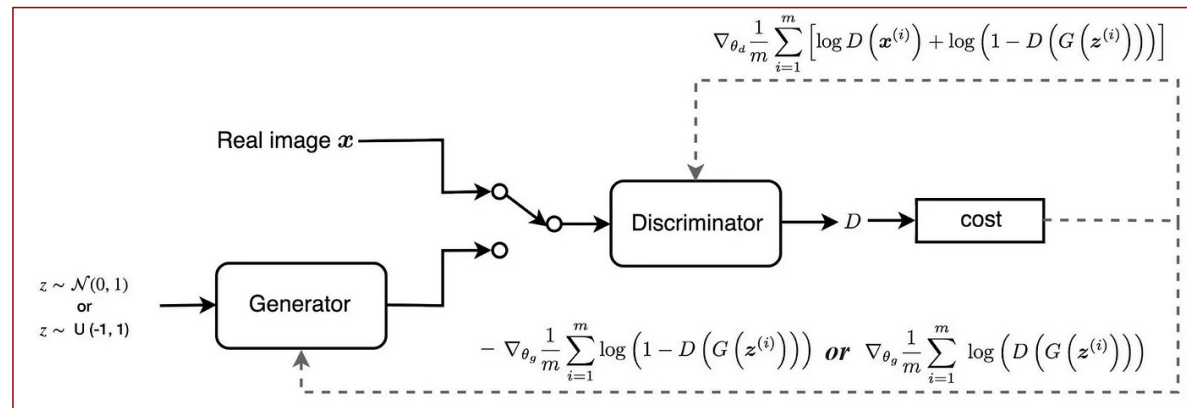
**Require:** :  $w_0$ , initial critic parameters.  $\theta_0$ , initial generator's parameters.

```
1: while  $\theta$  has not converged do
2:   for  $t = 0, \dots, n_{\text{critic}}$  do
3:     Sample  $\{x^{(i)}\}_{i=1}^m \sim \mathbb{P}_r$  a batch from the real data.
4:     Sample  $\{z^{(i)}\}_{i=1}^m \sim p(z)$  a batch of prior samples.
5:      $g_w \leftarrow \nabla_w \left[ \frac{1}{m} \sum_{i=1}^m f_w(x^{(i)}) - \frac{1}{m} \sum_{i=1}^m f_w(g_\theta(z^{(i)})) \right]$ 
6:      $w \leftarrow w + \alpha \cdot \text{RMSPProp}(w, g_w)$ 
7:      $w \leftarrow \text{clip}(w, -c, c)$ 
8:   end for
9:   Sample  $\{z^{(i)}\}_{i=1}^m \sim p(z)$  a batch of prior samples.
10:   $g_\theta \leftarrow -\nabla_\theta \frac{1}{m} \sum_{i=1}^m f_w(g_\theta(z^{(i)}))$ 
11:   $\theta \leftarrow \theta - \alpha \cdot \text{RMSPProp}(\theta, g_\theta)$ 
12: end while
```

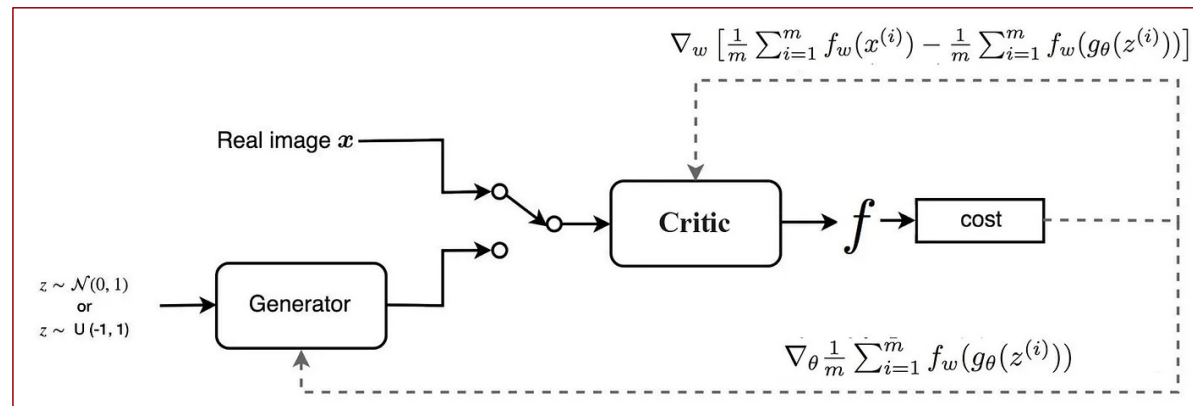
---

# GAN vs. WGAN

GAN



WGAN



# GAN: Applications

---



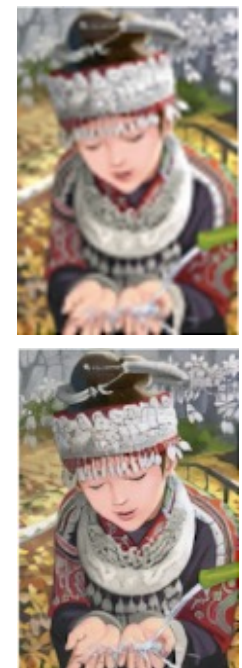
Image-to-Image translation



CycleGAN



Inpainting



Super-resolution

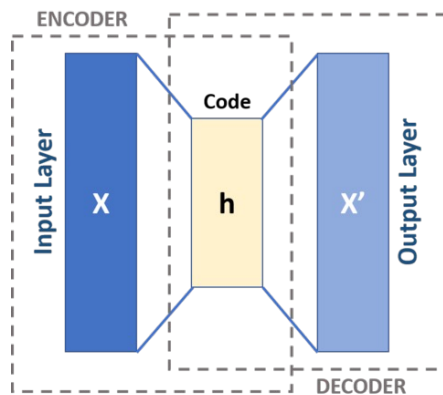
# Summary: GAN

---

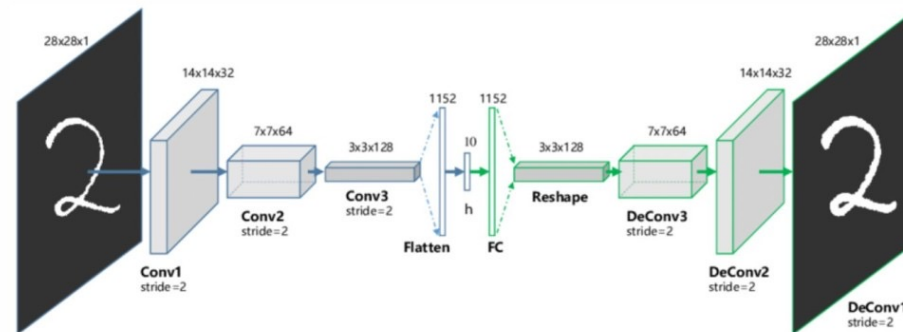
- Pros:
  - High-quality data generation especially in domains like image synthesis, video generation, and audio creation.
  - Capability of learning complex data distributions and applying to a wide variety of tasks, including image-to-image translation, inpainting, super-resolution, and more.
  - Augmenting deep learning by generating synthetic training data.
- Cons:
  - GANs can be notoriously difficult to train: instability, sensitivity to hyperparameters, mode collapse, etc.

# Autoencoder

- An autoencoder is a type of neural network used for unsupervised learning tasks that aims to learn efficient representations of data.
  - Consisting of an encoder-decoder architecture, autoencoders transform input data into a compact representation and then attempt to reconstruct the original data from this compressed form.



Scheme of autoencoder



Example of convolutional autoencoder

# Autoencoder

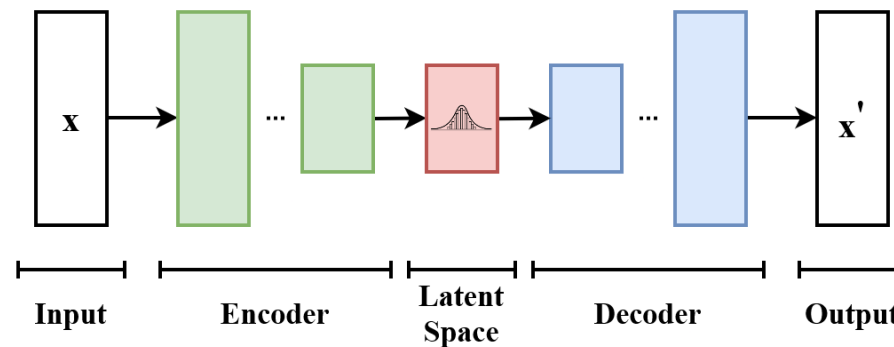
---

- Encoder:
  - The encoder compresses the input data into a latent-space representation or code, often of lower dimension than the input.
  - It is typically implemented as a series of layers that progressively reduce the dimensionality of the input.
- Latent Space:
  - The latent space, or bottleneck, is where the most compact representation of the data is stored.
  - This encoded representation captures the essential features necessary to reconstruct the data.
- Decoder:
  - The decoder reconstructs the data from the latent representation.
  - Its structure often mirrors the encoder, progressively increasing the dimensions back to those of the original input.

# Variational Autoencoder (VAE)

---

- VAE introduces a probabilistic approach to latent space representation.
  - The encoder learns a distribution, i.e., outputting the mean and the standard deviation of many inputs, instead of mapping to a single point in the latent space.
  - Sampling is performed in the latent space.
  - Decoder reconstructs samples based on new sampling in the latent space.



# VAE: Usages

---

- Data generation
  - VAEs are used for generating new samples that resemble the training data, such as in image, text, or audio generation.
- Anomaly detection
  - By learning the typical distribution of the training data, VAEs can identify outliers that deviate significantly from this norm.
- Semi-supervised learning
  - VAEs can be adapted for semi-supervised learning tasks, leveraging the generative capability to improve learning with limited labeled data.
- Representation learning
  - The learned latent space in VAEs can be used for better feature representations that enhance downstream tasks like classification or clustering.

# Summary

---

- CNNs are a class of deep neural networks effective for image and spatial data processing, leveraging convolutional layers to automatically learn hierarchical features.
- Other than discriminative learning, deep generative learning focuses on modeling the distribution of data to generate new data samples.
  - GAN uses adversarial training between a generator and a discriminator.
  - VAE employs a probabilistic framework to encode and decode inputs through a latent space.